

Part I - Q1

SQL uses NULL to indicate unavailable or unknown information. Because NULL does not represent an actual value, comparisons such as 'phone_number = NULL' never evaluate to TRUE; they produce UNKNOWN. Since the WHERE clause keeps only rows that evaluate to TRUE, records with missing phone numbers are excluded and COUNT(*) returns 0.

Correct Query:

```
SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;
```

Most aggregate functions intentionally ignore NULL values. Including unknown data in calculations could lead to ambiguous or invalid results.

Q2

Yes, the student's argument is valid. GROUP BY department already collapses all records belonging to the same department into a single result row. As a consequence, duplicate department entries cannot appear in the output. Applying DISTINCT therefore has no effect and simply makes the query unnecessarily verbose.

Q3

A larger result from LEFT JOIN indicates that some customers do not have corresponding entries in the Order table. INNER JOIN displays only customers with matching order records, whereas LEFT JOIN preserves every customer regardless of whether a match exists.

For instance, if 500 customers exist but only 380 have placed orders, INNER JOIN will show 380 rows. LEFT JOIN, however, will return all 500 rows, assigning NULL values to order-related columns for customers who never ordered.

Q4

Standard SQL requires every selected attribute either to participate in grouping or to be aggregated. Since employee_name is neither grouped nor aggregated, the query is invalid.

A department may contain several employees, so there is no unique employee_name associated with the computed average salary. Returning an arbitrary name would produce misleading results.

Correct Query:

```
SELECT department, AVG(salary) FROM Employee GROUP BY department;
```

Q5

Aggregate expressions cannot be evaluated inside WHERE because grouping has not yet occurred at that stage. SQL logically processes statements in the sequence: FROM, WHERE, GROUP BY, HAVING, SELECT, and ORDER BY.

Since AVG(salary) is calculated only after grouping, the condition must be moved to HAVING.

Correct Query:

```
SELECT department, AVG(salary) FROM Employee GROUP BY department HAVING  
AVG(salary) > 80000;
```

Q6

(a) To identify products priced above the overall average, a non-correlated subquery is sufficient because the average price is computed once for the entire table.

```
SELECT * FROM Product WHERE price > (SELECT AVG(price) FROM Product);
```

(b) Comparing an employee's salary with the average salary of that employee's department requires a correlated subquery. The average depends on the department of the current employee, so the subquery must reference the outer query.

```
SELECT e1.employee_name, e1.department, e1.salary FROM Employee e1 WHERE e1.salary  
> (SELECT AVG(e2.salary) FROM Employee e2 WHERE e2.department = e1.department);
```

Non-correlated subqueries execute a single time, whereas correlated subqueries may execute repeatedly—once for each outer row—making them less efficient on large datasets.

Part II

(a) LEFT JOIN ensures that every customer appears in the result set. Selecting rows where o.order_id IS NULL isolates customers who have never placed an order. An INNER JOIN would fail to show these customers because unmatched rows are discarded.

(b) The calculation is performed in two stages. First, the total value of each order is computed. Next, those totals are averaged restaurant-wise. Since the filter depends on grouped data, HAVING is the appropriate clause.

(c) The overall platform average is a single fixed value. Computing it once through a non-correlated scalar subquery is more efficient than recalculating it separately for every restaurant.

(d) The correlated subquery determines the highest quantity sold within a particular restaurant. The HAVING condition then selects every menu item whose sales equal that maximum, ensuring ties are also returned.

A limitation of using GROUP BY with MAX() is that MAX() provides only the highest value, not the entire record associated with it. Window functions are generally more suitable for top-N analyses.

Part III

Functional Dependencies:

$\text{student_id} \rightarrow \text{student_name}$

$(\text{student_id}, \text{course_id}) \rightarrow \text{grade}$

$(\text{student_id}, \text{course_id}) \rightarrow \text{enrollment_date}$

$\text{course_id} \rightarrow \text{course_name}$

$\text{course_id} \rightarrow \text{instructor_id}$

$\text{instructor_id} \rightarrow \text{instructor_name}$

$\text{instructor_id} \rightarrow \text{instructor_office}$

The relation satisfies 1NF because every field stores indivisible values and no repeating attribute groups are present.

The schema suffers from partial dependencies because several attributes depend only on `student_id` or `course_id` instead of the complete composite key. This creates update, insertion, and deletion anomalies.

A transitive dependency also exists: $\text{course_id} \rightarrow \text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$. Consequently, modifying instructor details may require changes in multiple rows.

3NF decomposition:

Student(`student_id` PK, `student_name`)

Course(`course_id` PK, `course_name`, `instructor_id` FK)

Instructor(`instructor_id` PK, `instructor_name`, `instructor_office`)

Enrollment(`student_id` FK PK, `course_id` FK PK, `grade`, `enrollment_date`)

Part IV

Although NOT IN could solve the 'customers with no orders' problem, the presence of NULL values in the subquery result can lead to unexpected behaviour. LEFT JOIN combined with a NULL check avoids this issue and is therefore safer.

Normalization minimizes redundancy and prevents anomalies, but it also increases the

number of joins required to answer queries. For performance reasons, many production systems intentionally maintain some denormalized structures, especially in analytical and reporting environments.